Image Processing - Lecture 7

Amir Atapour-Abarghouei

amir.atapour-abarghouei@durham.ac.uk

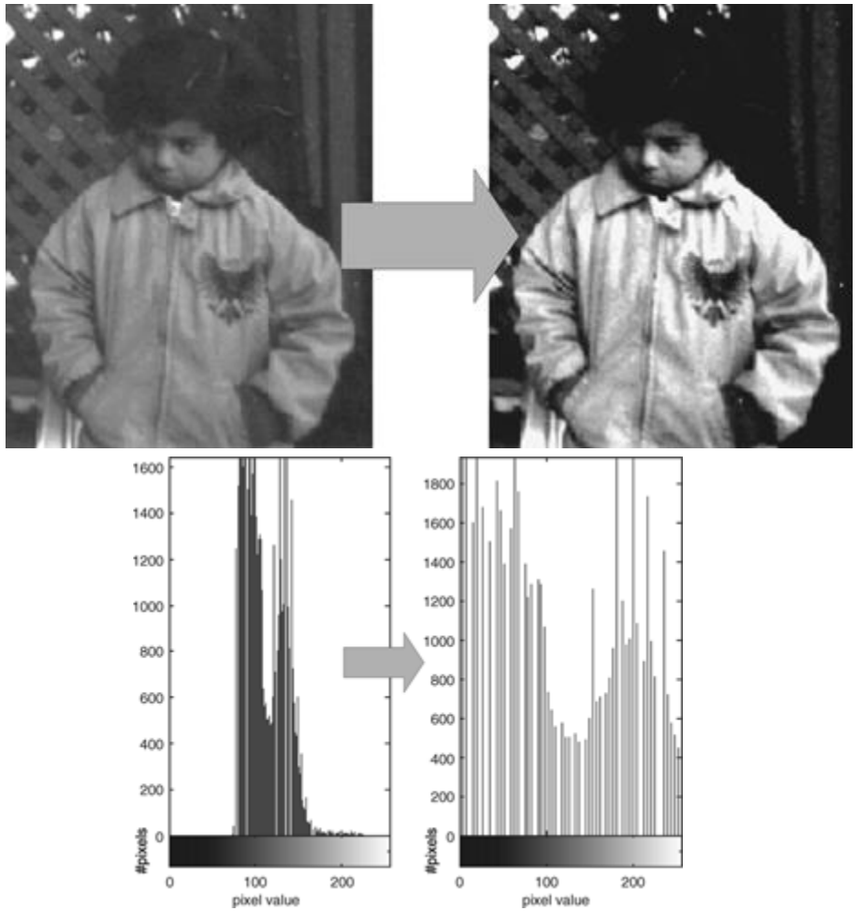
Reading:

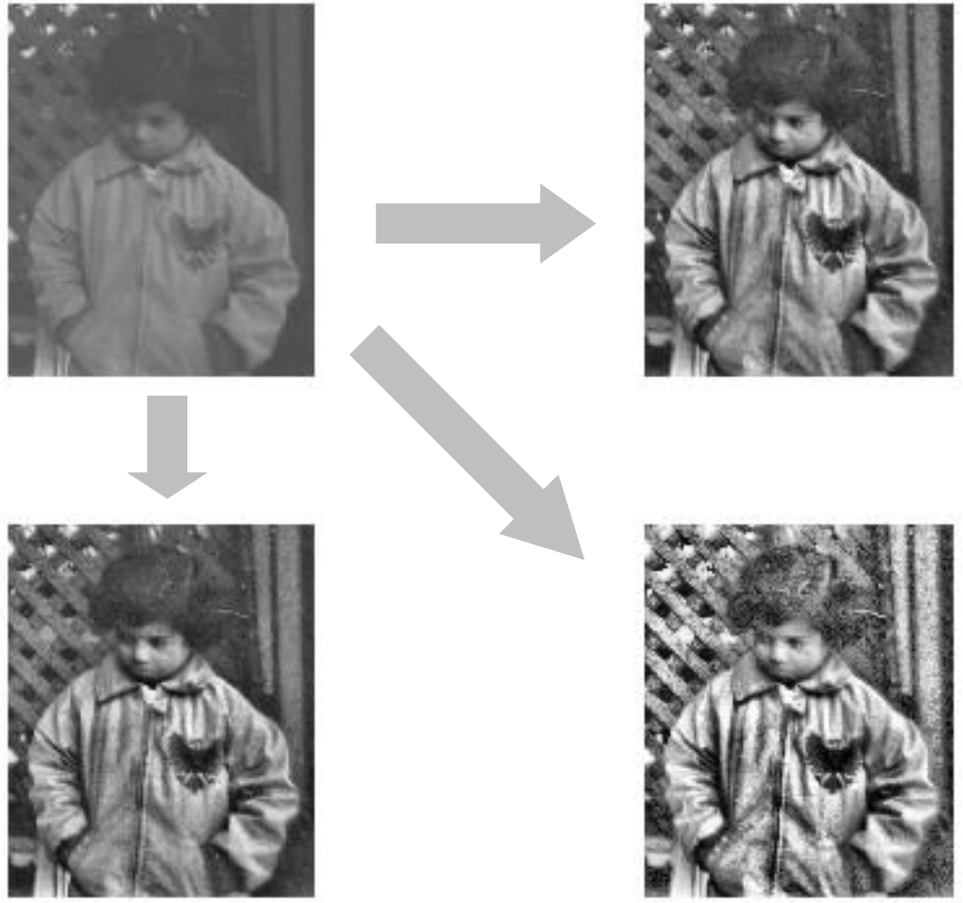
Szeliski - Section 3.4
Gonzalez/Woods - Chapter 4
Solomon/Breckon - Chapter 5

Images in Fourier Representation: Fourier Space I

Slide material acknowledgements: Narasimhan, Bebis, Fisher; Breckon; Ivrissimtzis; Atapour

Recap ... Histograms





Previously: images as statistical distributions and how to manipulate them.

Today:
Images in Fourier Space

The Basics

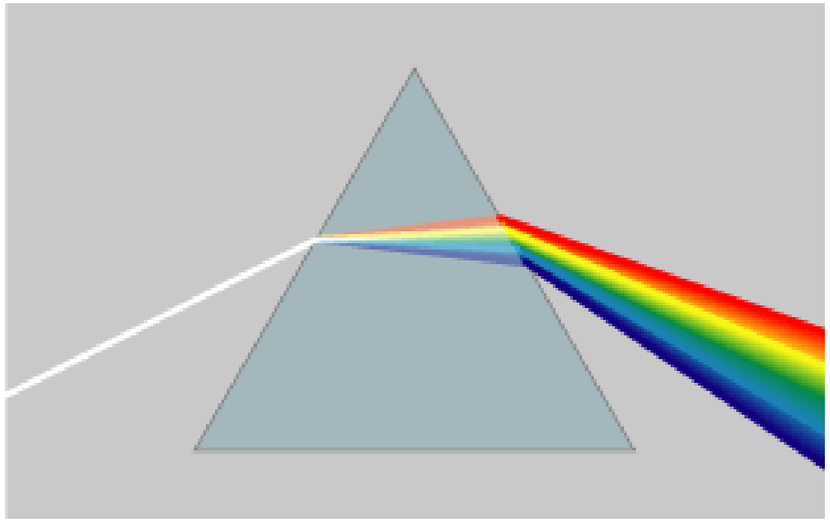
- The **Fourier transform** of an image is a global transform.
- Produces a representation of the image in Fourier space, (or frequency domain).
  - **Perform operations that are not trivial in the spatial domain!**
- Performed by applying the **Discrete Fourier Transform** (DFT) operator:
  - decomposes image into frequency components (sinusoidal functions)
  - the frequency components are arranged in an array of the same size as the image (sometimes it is convenient to think of the output of the DFT as an image, even though **'pixel values' are complex numbers**).
  - major applications in image processing, analysis, compression, ….

Terminology - Domain

- **Spatial (or Real) Domain:** images (signals) are represented by a spatial layout of samples that are real numbers
  - i.e. pixel intensities for images (2D)
  - i.e. samples for signals (1D).
- **Frequency (or Fourier) Domain:** where images (signals) are represented by coefficients of sinusoidal (sines, cosines) basis frequencies
  - 2D spatial frequencies for images
  - 1D waveform frequencies for signals.

Fourier series: Any periodic function can be rewritten as a weighted sum of Sines and Cosines of different frequencies.

The Fourier Transform: An analogy



A useful **analogy to the Fourier Transform** is a glass prism.

A glass prism is a physical device that separates light into various colours based on its wavelength (or frequency) content.

The Fourier transform → a **'mathematical prism'** that **separates a function [or image] into components** based on frequency content.

- Gonzalez / Woods 2001

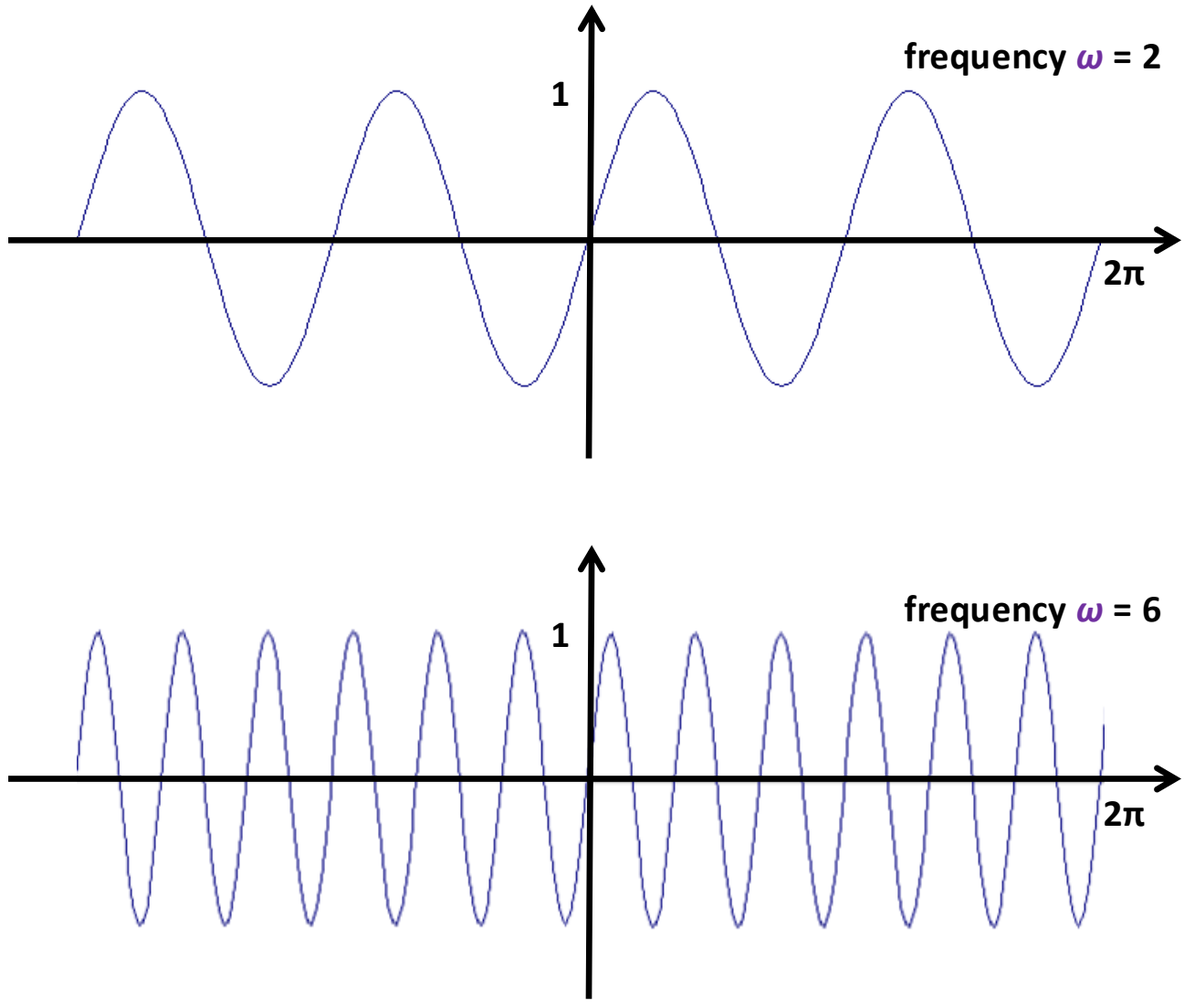
Jean Baptiste Joseph Fourier (1768-1830)

- Had an idea in 1807:
  - Any periodic function can be rewritten as a weighted sum of Sines and Cosines of different frequencies.
- Don't believe it?
  - Neither did Lagrange, Laplace, Poisson or any of the other big wigs!
  - Not translated into English until 1878!
- But it is true!
  - called **Fourier Series**
  - Possibly the greatest tool in Engineering.

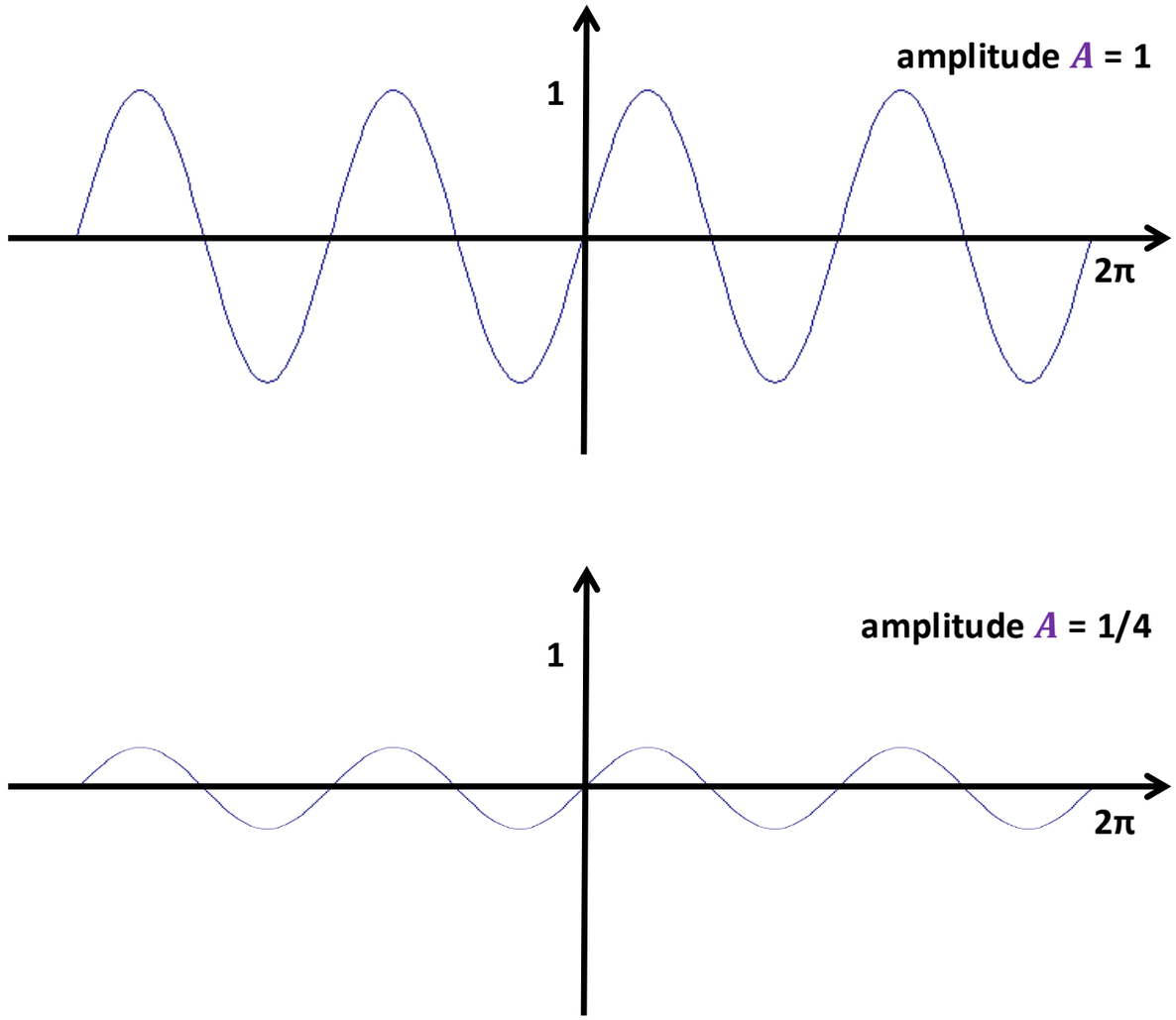
To illustrate the main point, we first consider a simple 1D signal.

(think of it as a sound wave)

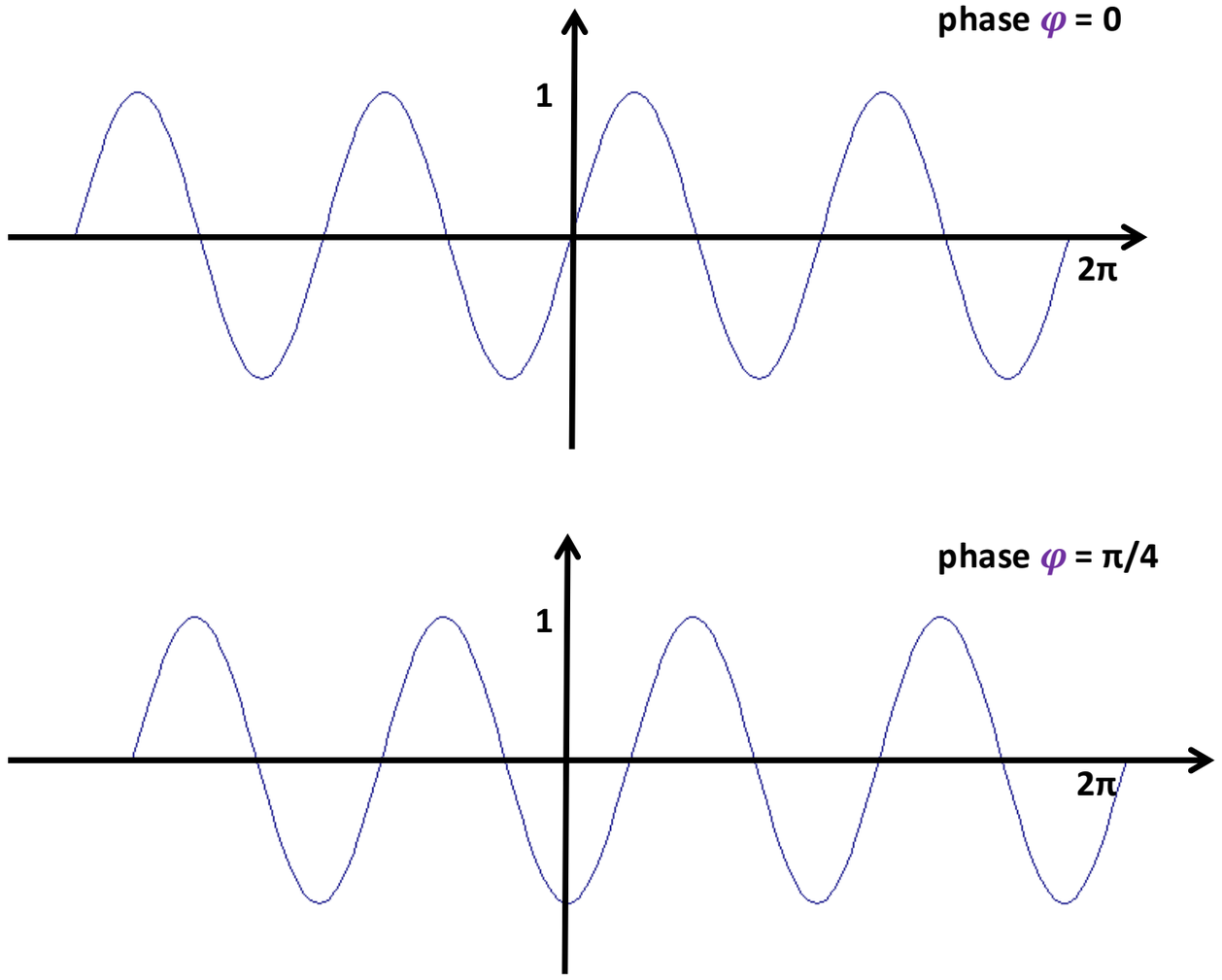
Reminder: Frequency



Reminder: Amplitude



Reminder: Phase



1D Fourier Series

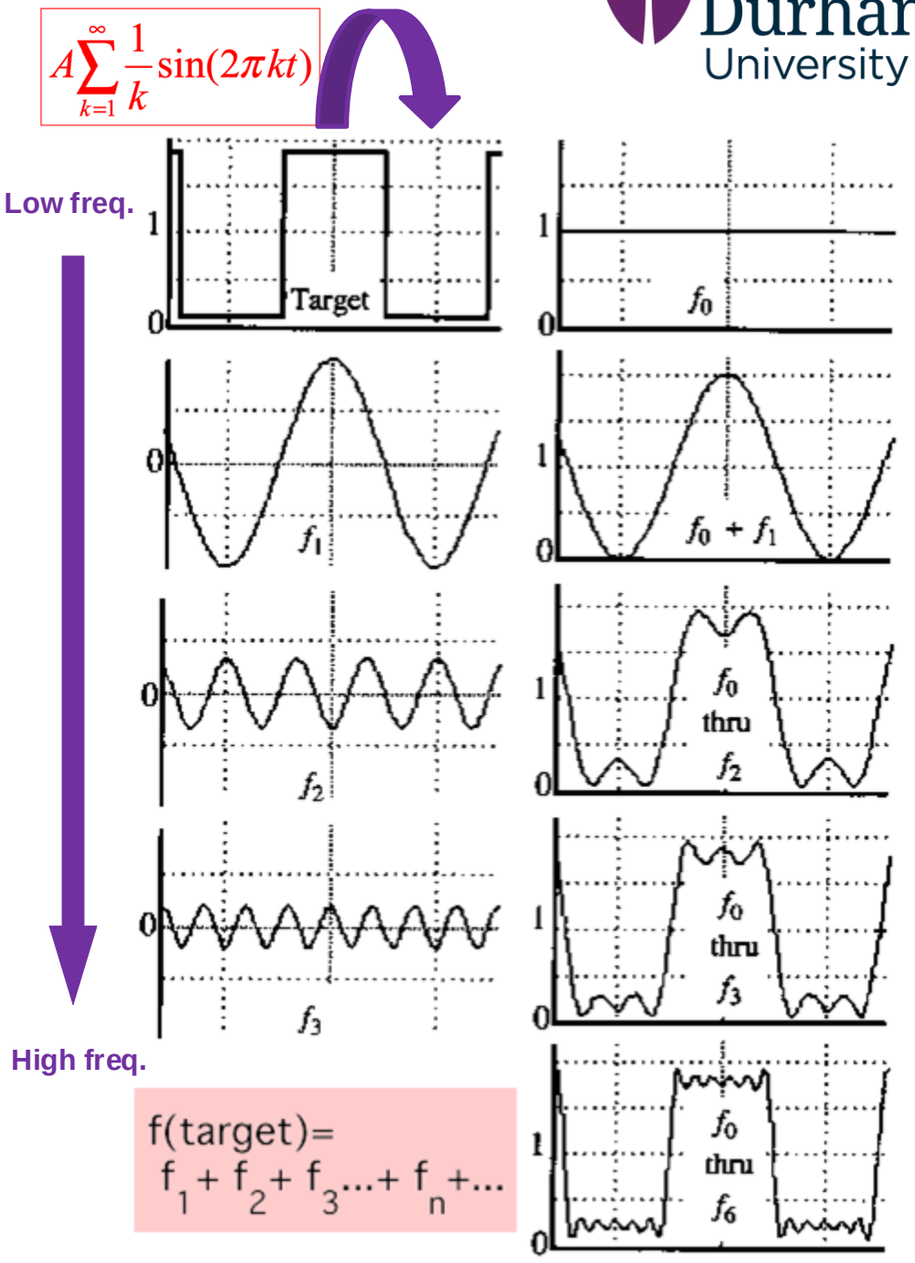
- Our building block:

$$A \cdot sin(\omega \cdot x + \varphi)$$

- Add enough of them to get any signal $f(x)$

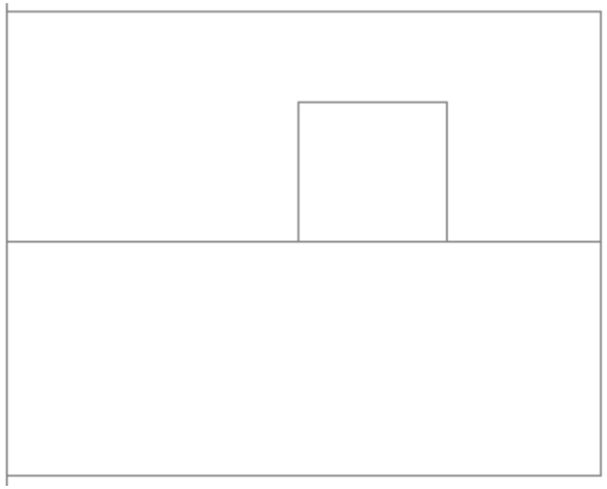
$$f(x) = f_1(x) + f_2(x) + f_3(x) + \cdots$$

  - ω is the frequency. For each frequency, we have one sinusoidal function in the sum.
  - A is the amplitude. It is the weight of that sinusoidal function in the sum.
  - φ is the phase. Changes in the phase shift a sinusoidal function to the left or right.



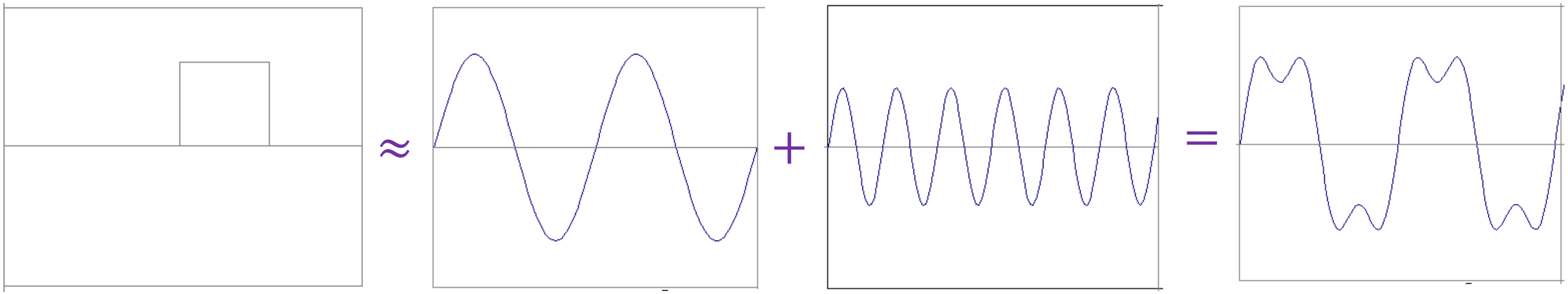
1D Fourier Series

- Increasing the number of frequency components allows **any signal waveform** to be created.
  - higher frequency components contribute less to the coarse outline of the signal and more to its fine details.

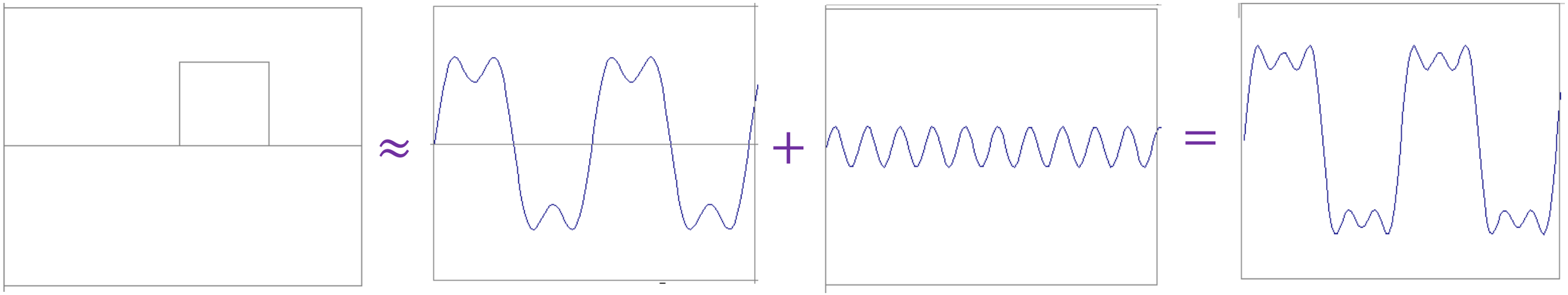


$$= A\sum_{k=1}^{\infty}\frac{1}{k}\sin(2\pi kt)$$

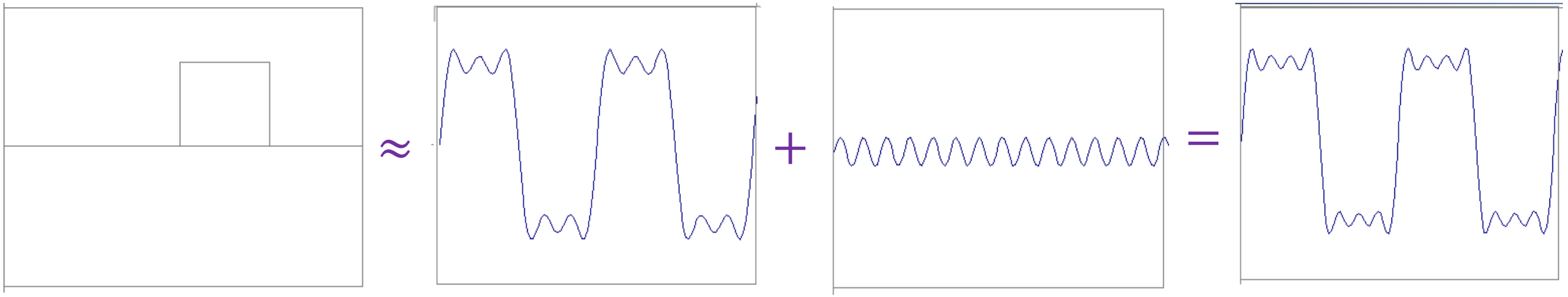
Example 1D Signal: Square Wave



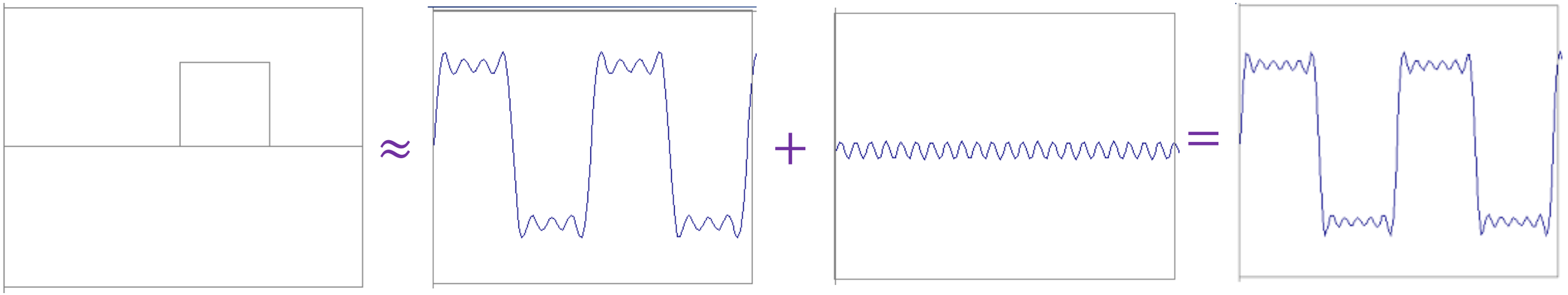
Example 1D Signal: Square Wave



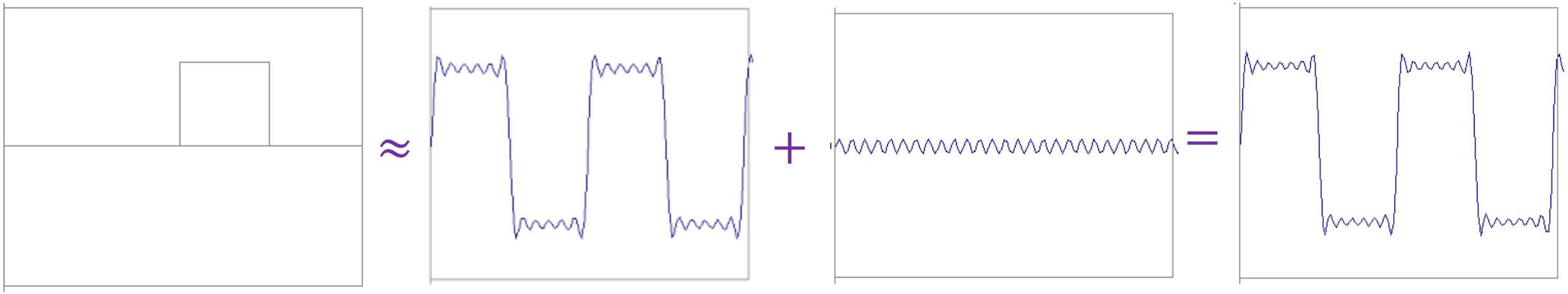
Example 1D Signal: Square Wave



Example 1D Signal: Square Wave



Example 1D Signal: Square Wave

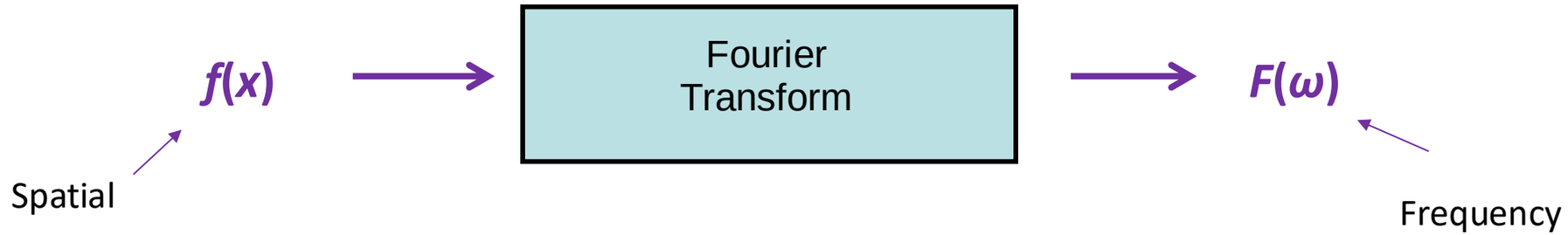


Low Frequency components influence the coarse outline of the signal.

Higher Frequency components influence the fine detail of the signal.

Fourier Transform

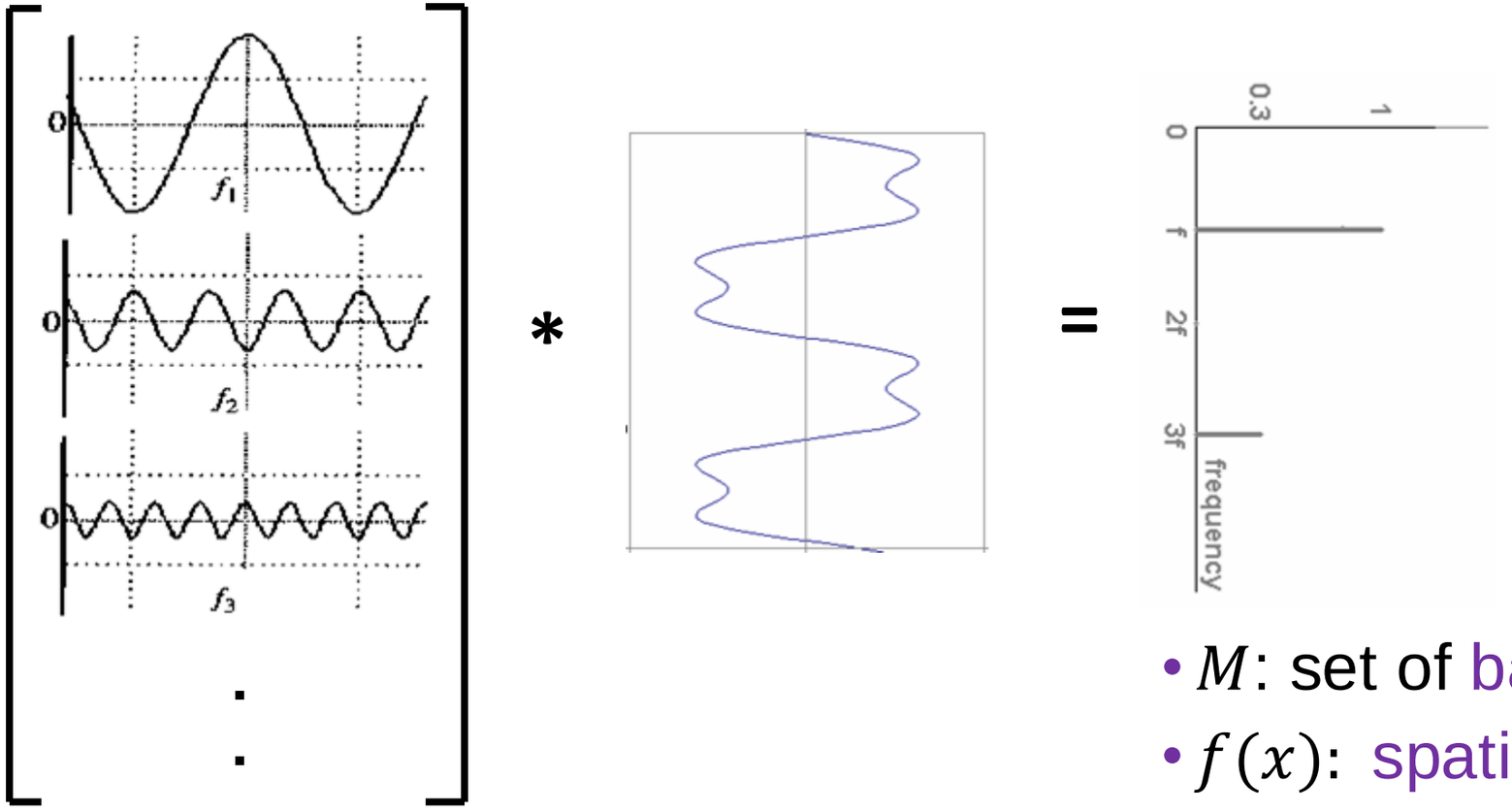
We want to understand the signal as a sum of weighted and phase shifted frequencies. The Fourier transform reparametrises the signal by ω instead of x:



$F(\omega)$ itself is called the Fourier transform of $f(x)$. The inverse Fourier transform applied on $F(\omega)$ parametrises the signal again by x:

1D Discrete Fourier Transform

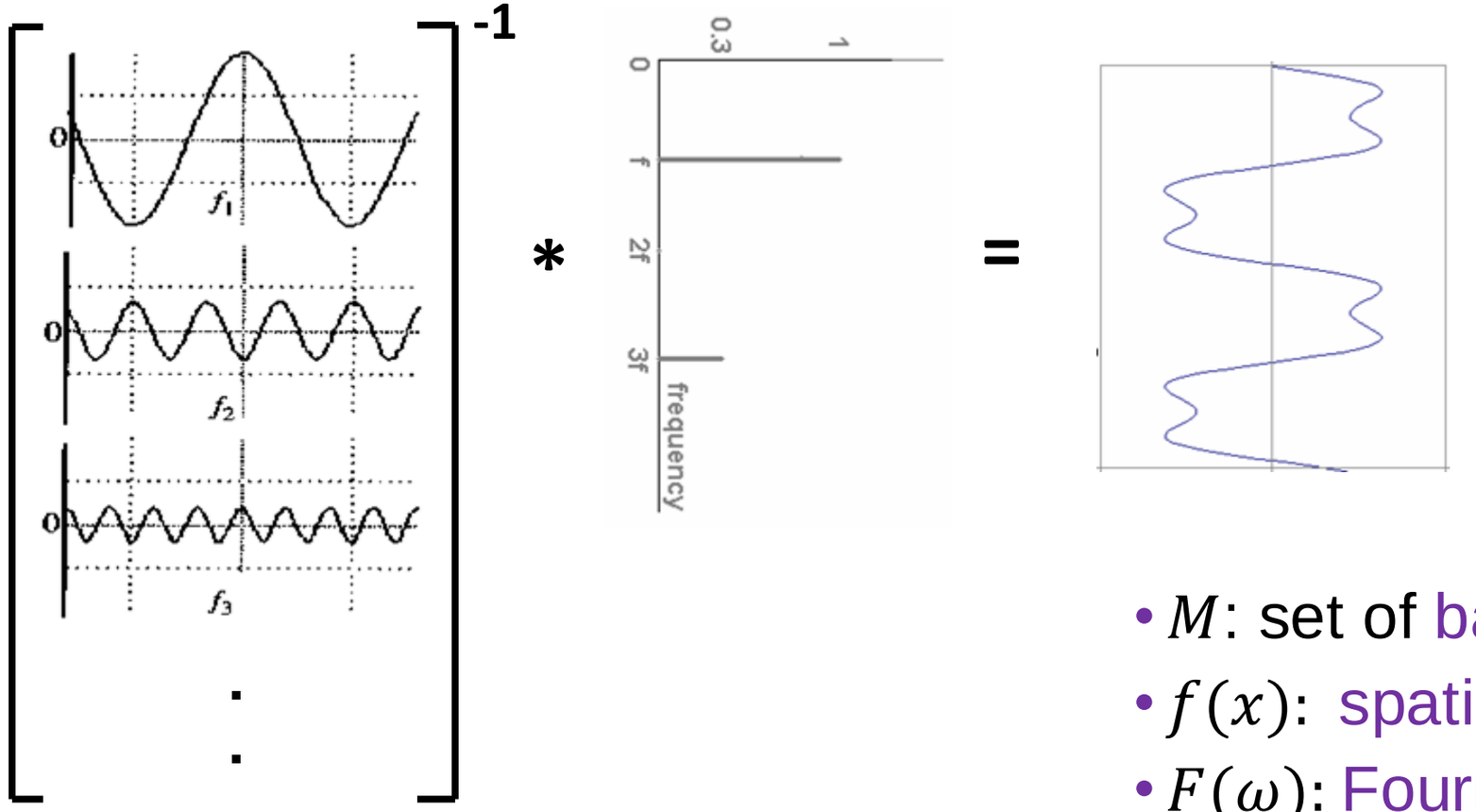
The Fourier transform $M * f(x) = F(\omega)$ is a change of mathematical basis.



- M: set of basis functions.
- $f(x)$: spatial domain.
- $F(\omega)$: Fourier domain representation.

1D Discrete Fourier Transform

The **inverse** Fourier transform $M^{-1} * F(\omega) = f(x)$ is again a change of basis.



- M: set of basis functions.
- $f(x)$: spatial domain.
- $F(\omega)$: Fourier domain representation.

Complex Numbers in Fourier Transform

For frequency ω, the Fourier transform at that point $F(\omega)$ is defined by amplitude A and the phase φ of the corresponding sine.

- *How can F hold both? Complex number trick!*

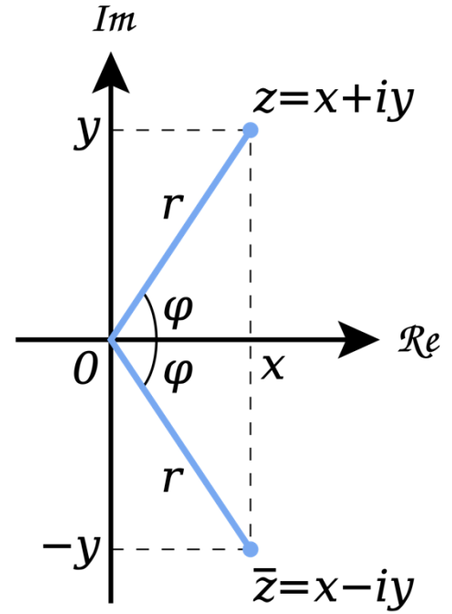
- While we can handle amplitude and phase separately as two real functions, it is mathematically convenient to use complex numbers.

In your implementations, when you call the Fourier transform through a library function, be prepared to get an output of complex numbers.

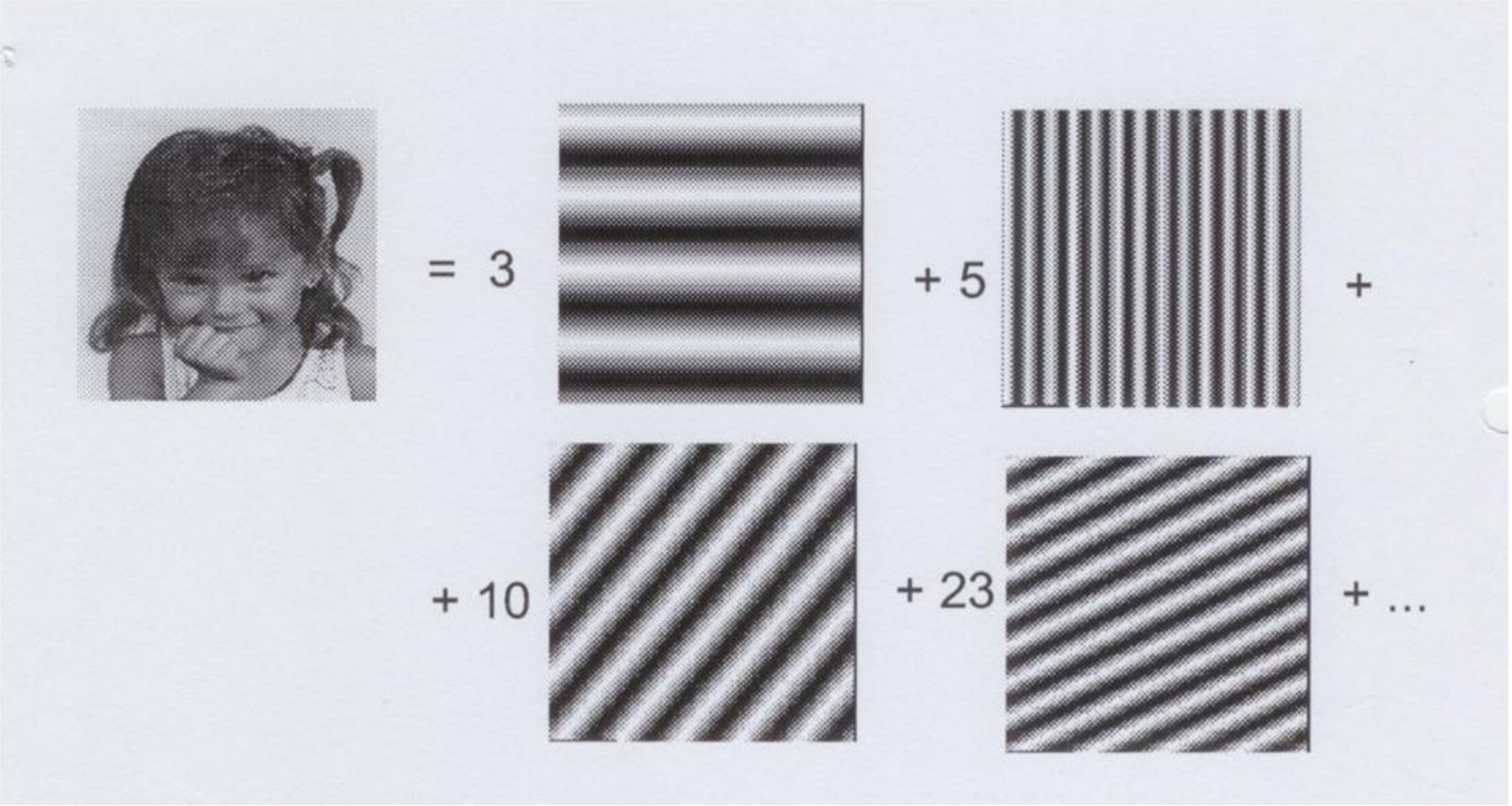
$$\boldsymbol{F(\omega) = R(\omega) + iI(\omega)}$$

$$A = \pm\sqrt{R(\omega)^2 + I(\omega)^2}$$

$$\varphi = \tan^{-1}\frac{I(\omega)}{R(\omega)}$$

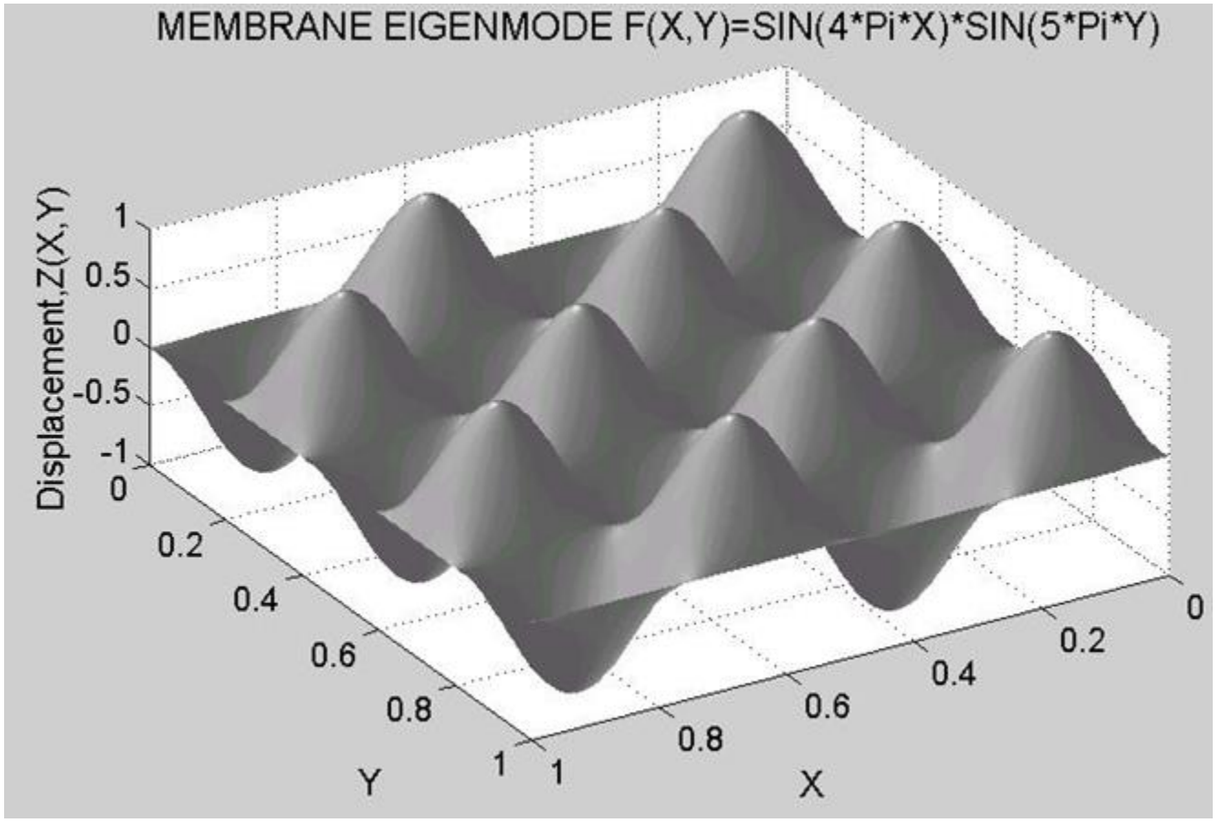


Fourier Transforms of 2D Images

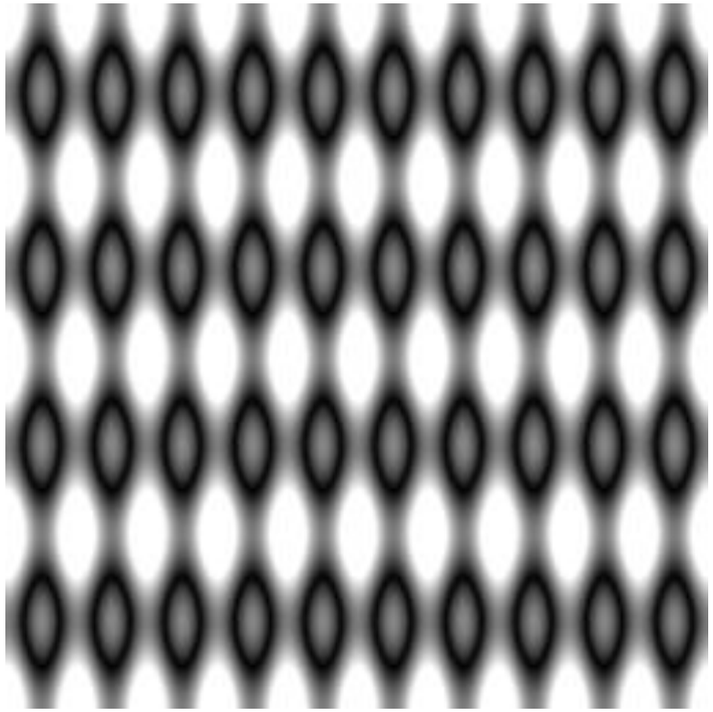


Fourier transform for 2D images using 2D sine waves as basis functions.

2D Sine Functions - Examples



3D plot of $F(X,Y)$ as a two variable function.



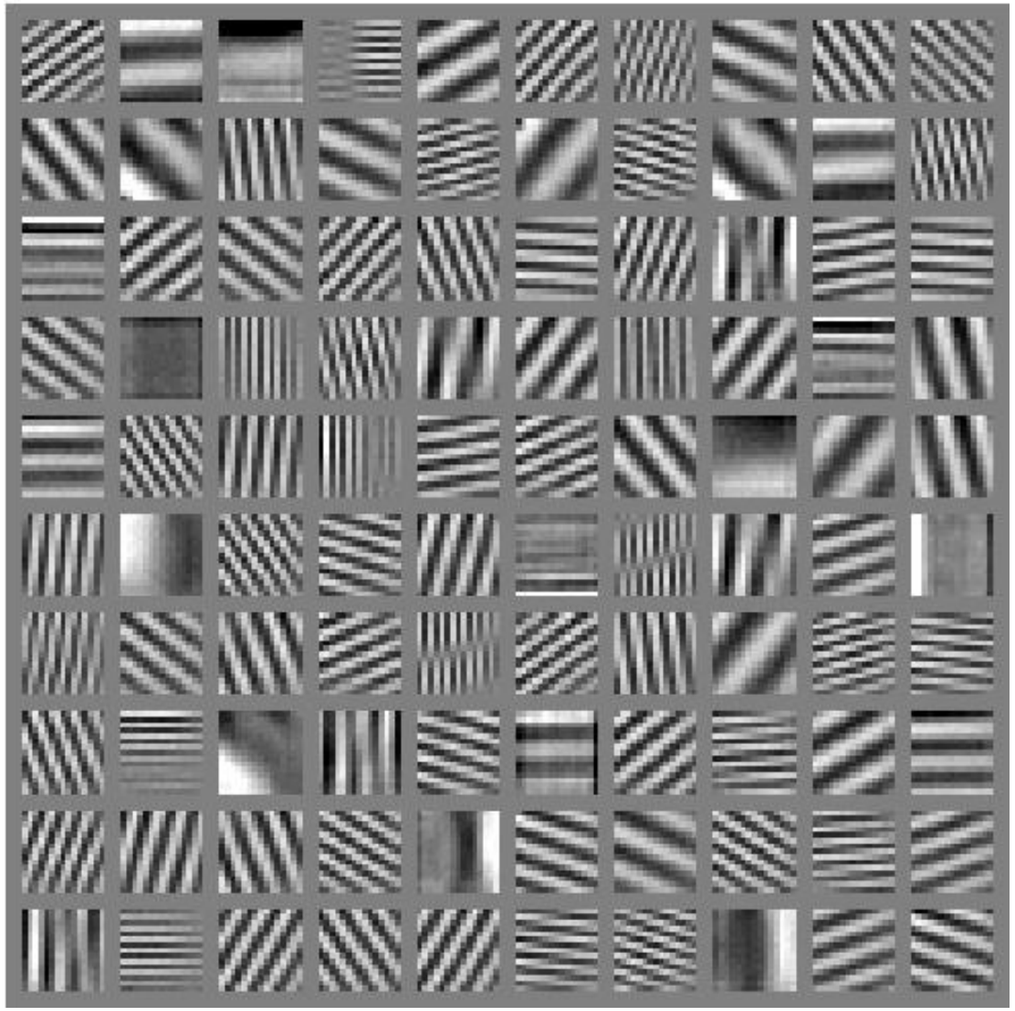
2D image of $F(X,Y)$
with function values as
greyscale intensity values.

2D Sine Functions

Examples of a subset of frequency components.

Any *image can be reconstructed as a weighted sum of different frequencies aligned in different phases.*

weights = Fourier coefficients of the image.

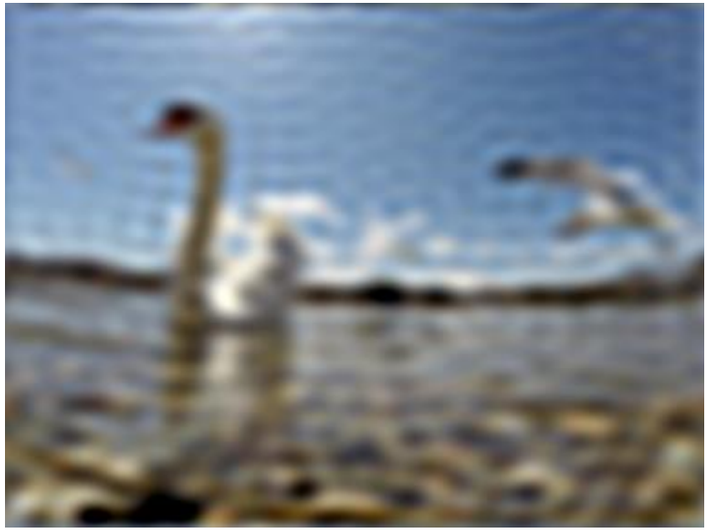


How the DFT matrix is constructed: https://en.wikipedia.org/wiki/DFT_matrix
[beyond the scope of this module]

Image Frequencies

Original

20 highest frequencies (only)



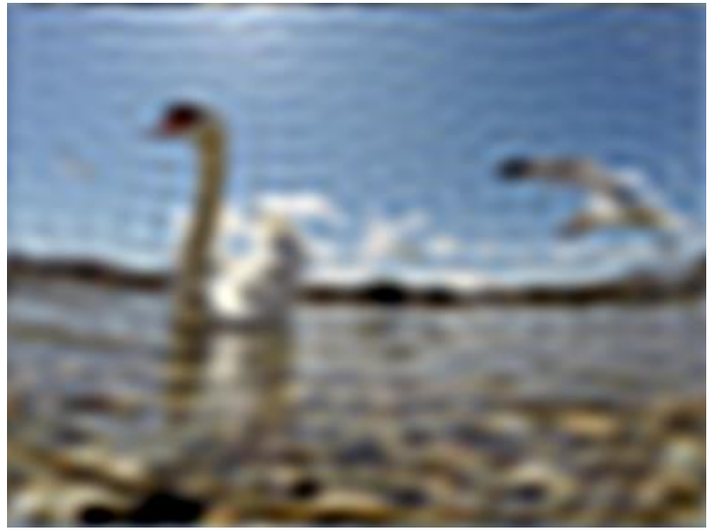
20 lowest frequencies (only)

Image Frequencies

High frequency components influence the finer detail of the image (edges).

20 highest frequencies (only)

Low frequency components influence the overall shape of the image (coarse outline).



20 lowest frequencies (only)

Discrete Fourier Transform

The Fourier transform operator we apply on images is called Discrete Fourier Transform (DFT).

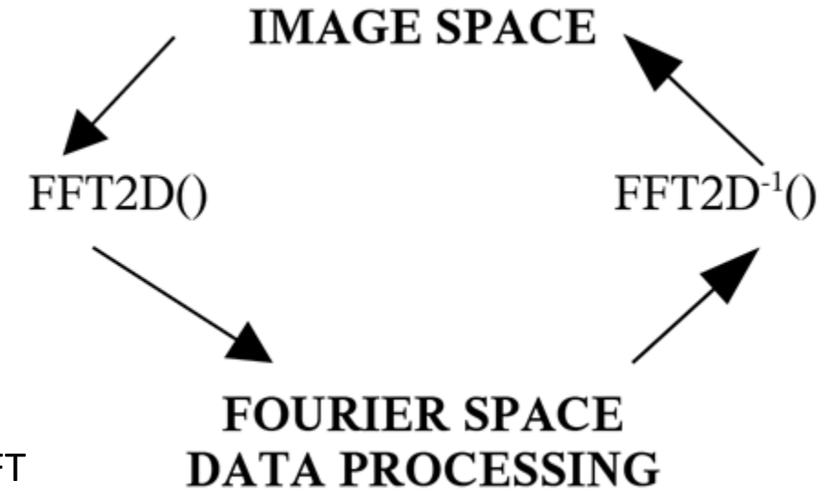
Inverted by the Inverse Discrete Fourier Transform (DFT^{-1}).

Why "discrete"? Because of the discrete nature of image sampling (pixels).

Computable efficiently via 2D Fast Fourier Transform (FFT)

- 1D FFT is $O(Nlog_2N)$
- 2D FFT is a series of $2N$ 1D FFTs
  - $O(N^2\log_2N)$
- *Algorithm details: beyond our scope*

matrix factorisation of the DFT into sparse matrices, which are then multiplied to obtain the DFT



Fast Fourier Transform – *Why Care?*

- **Fourier** Space **Filters** are **multiplicative** image operations
  - operate on the **Discrete Fourier Transform (DFT) of image**
    - **suppress** certain **frequencies** whilst leaving **others unchanged.**
    - **synonymous to filtering** in the frequency domain**.**
- Convolution operation in spatial domain (pixel space) (*Lecture 5*):

$$I_{output}(i,j) = \sum_{k=1}^{N}\sum_{l=1}^{M} I_{input}(i-(N-1)+k, j-(M-1)+l)m_{kl}$$

If S, P and F denote DFT's of I_{ouput}, m_{kl} and I_{input} respectively then **convolution** in **Fourier space** transforms to:

$$S = P\ F$$

*[Known as the **(Discrete) Convolution Theorem**]*

Discrete Convolution Theorem

- **Convolution** in the **frequency** domain (Fourier space) reduces to **multiplication** in the **spatial** domain (Real space).

and vice versa ....

- **Multiplication** in the **frequency** domain reduces to **convolution** in the **spatial** domain.

$$f_{\text{ouput}} = \sum\sum m\, I_{\text{input}}$$

Spatial Domain

$$\text{DFT}^{-1}(S = P\ F)$$

Frequency Domain

Discrete Convolution Theorem

- If $\boldsymbol{m_{kl}}$ is **small**:
  - compute convolution directly in the **spatial domain (real space).**

- But if $\boldsymbol{m_{kl}}$ is **large**:
  - more **efficient** to use a Fast Fourier Transform (FFT) to **obtain a Fourier domain version the image.**
  - then perform the **convolution-based filtering** as **multiplication** in **Fourier space.**

Visualising Fourier Images

- The DFT output (F_{nm}) of an image I_{input}
  - is a complex number valued image
  - contains the coefficients of the DFT of I_{input}
  - dimension are $n \times m$ (same as I_{input}).
- It can be displayed as the real and imaginary parts of the complex image

$$F_{nm} = G_{nm} + iH_{nm}$$

where

$$G_{nm} = \text{real part of } F_{nm}$$

$$H_{nm} = \text{imaginary part of } F_{nm}$$

Visualising Fourier Images

Components commonly used for visualising (seeing!) the Fourier spectrum.

– **Amplitude** (magnitude) spectrum:

$$|F_{nm}| = \sqrt{G_{nm}^2 + H_{nm}^2}$$

– **Phase** spectrum:

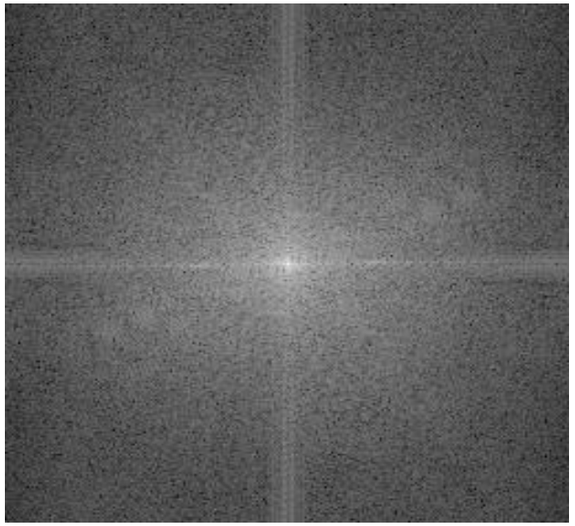
$$\varphi_{nm} = \tan^{-1}\left(\frac{H_{nm}}{G_{nm}}\right)$$

– **Power** spectrum (where $|\cdot|$ is the norm of a complex number):

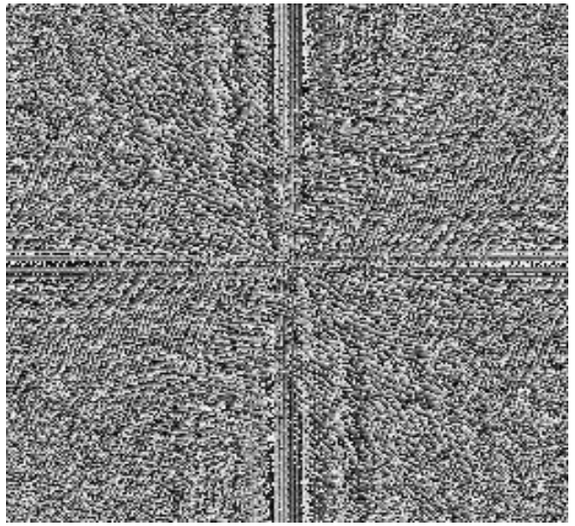
$$|F_{nm}|^2$$

A Fourier domain image has larger range than the spatial domain image. Normally calculated and stored using floating-point data types.

Visualising Fourier Spectrum



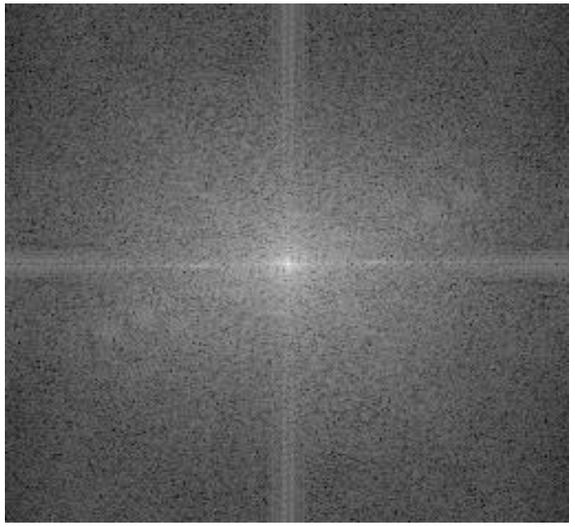
Magnitude
(logarithmically transformed)



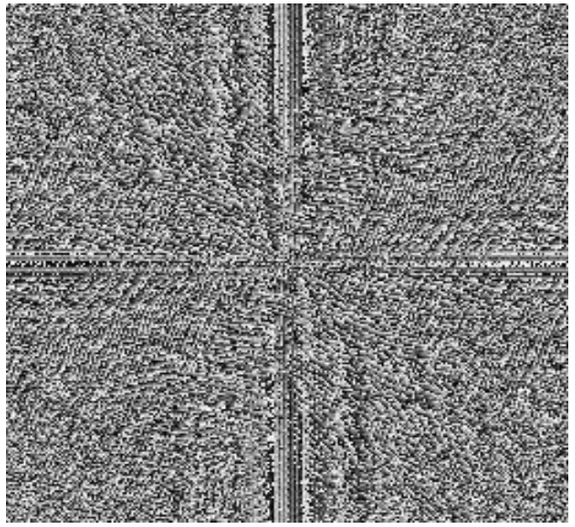
Phase

- transformed so image mean, $F(0,0)$, is centred (by convention)
  - DC-component, is at the centre == average brightness of the image.
  - The highest frequency present is $F(N-1, N-1)$.
- The magnitude is presented on a logarithmic scale as the floating point range is very large.

Understanding Fourier Spectrum



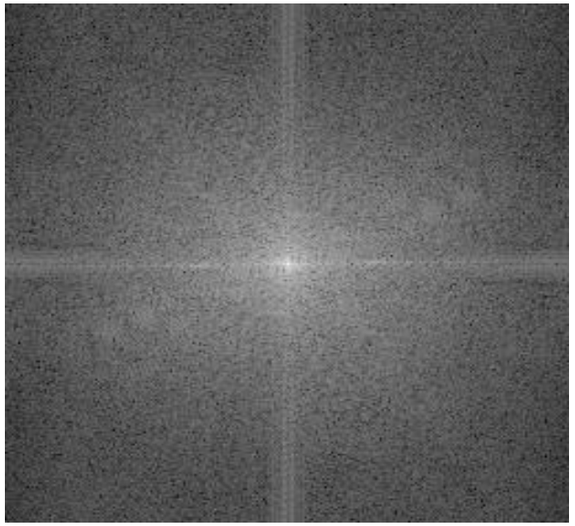
Magnitude
(logarithmically transformed)



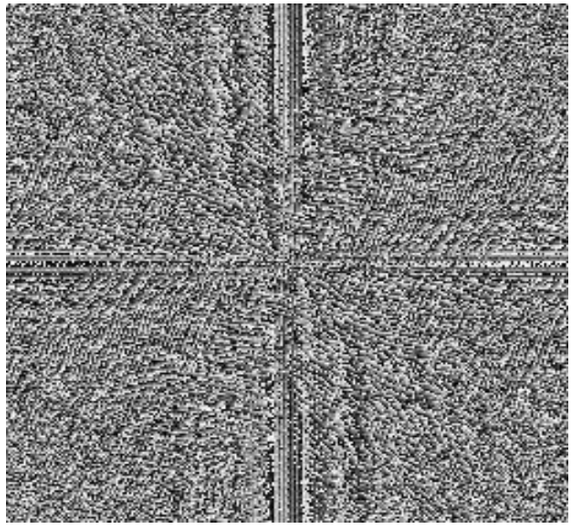
Phase

- Magnitude and phase images contain all frequencies.
  - Lower frequencies (close to centre of magnitude image) are larger than the higher frequencies (near the boundary of the magnitude image).
  - Thus, more information in lower frequencies.

Understanding Fourier Spectrum



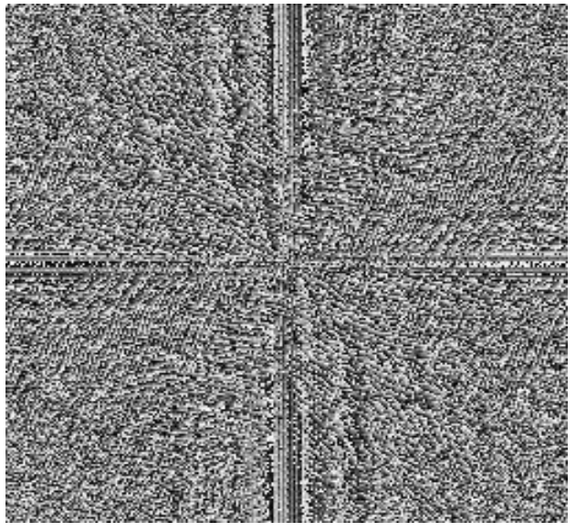
Magnitude
(logarithmically transformed)



Phase

- Two predominant directions in the magnitude image, *horizontal* and *vertical*, **correspond** to horizontally and vertically changing patterns of the image.

Understanding Fourier Spectrum

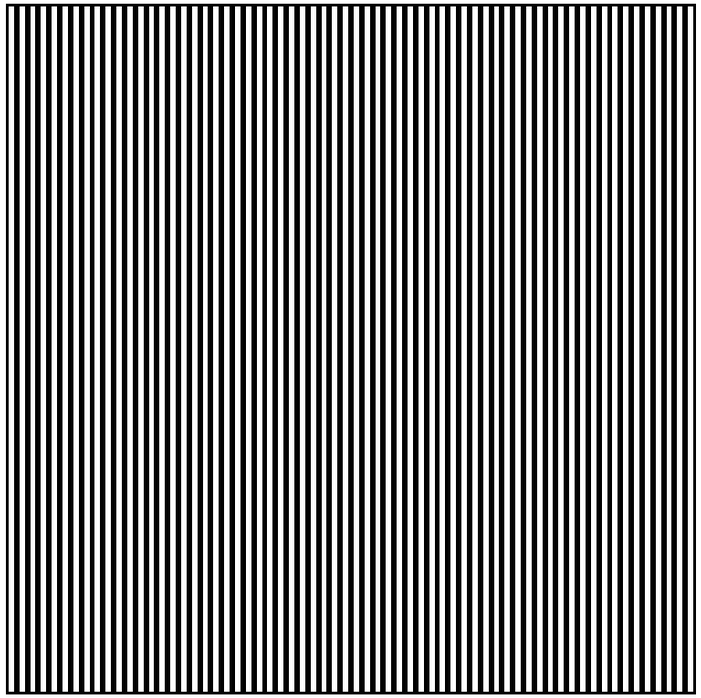


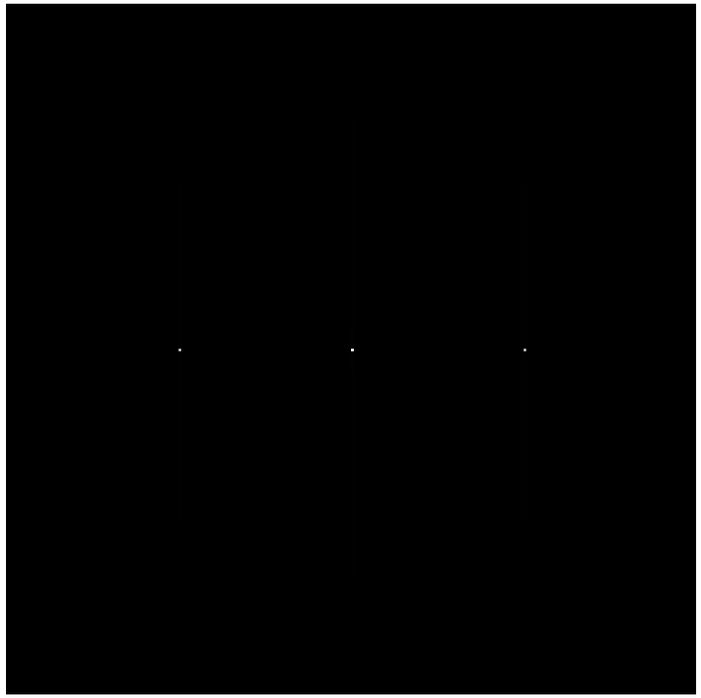
Phase

- The phase spectrum contains the phase part of the frequencies.
- Vertical and horizontal features correspond to image patterns.
  - not much new structural information about the original image.
- It is crucial in reconstructing the original via inverse Fourier transform.
  - **Signal reconstruction needs phases and amplitudes.** *[Corrupted otherwise]*

Understanding Fourier Spectrum

Simple Example



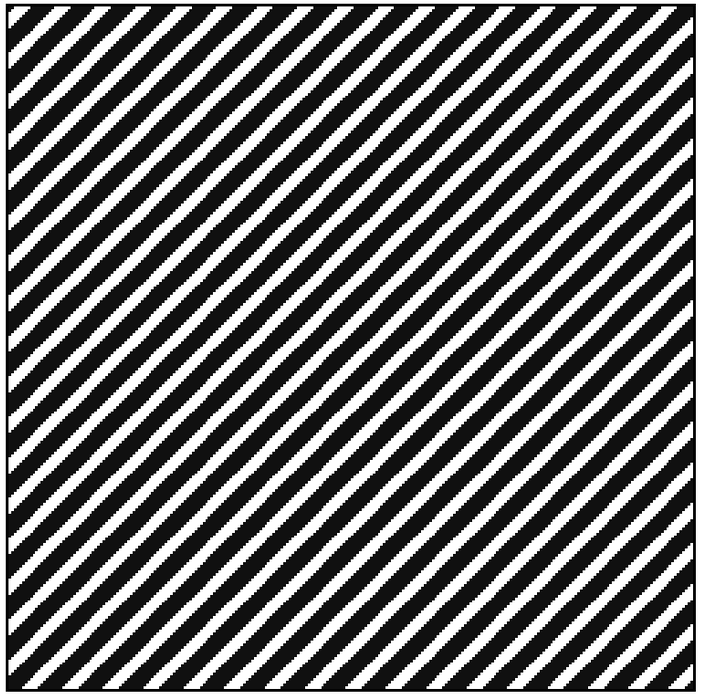


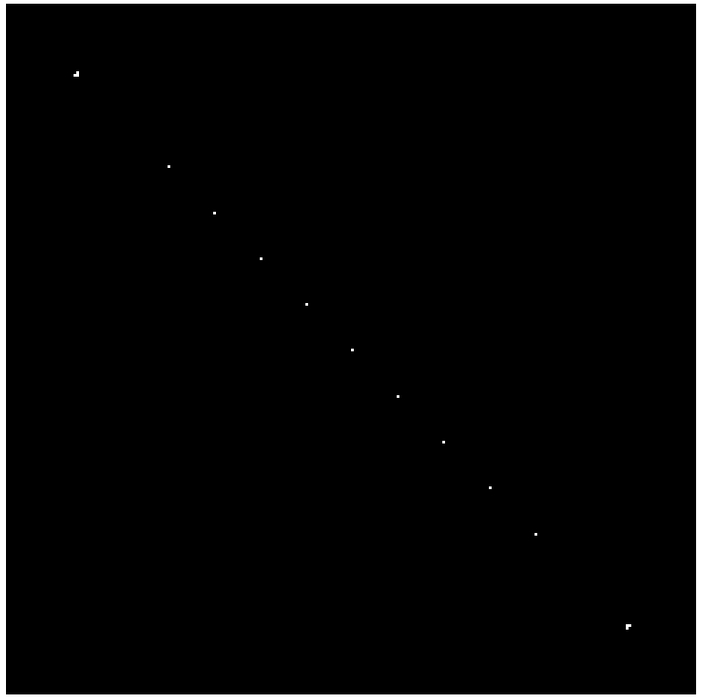
DFT Magnitude

- **3 main values:** DC value + 2 for frequency of input image.
- horizontal line - direction of greatest intensity change.

Understanding Fourier Spectrum

Simple Example





DFT Magnitude [Thresholded]

- all frequencies on same **diagonal:** main direction of variation.
- all at multiples of basic frequency – *harmonics.*

Predominant Directions in Magnitude Image

DFT Application: Orientation Correction

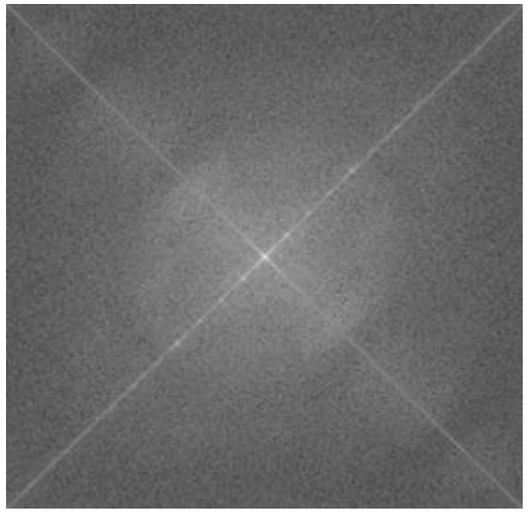
Application: scanner document orientation correction

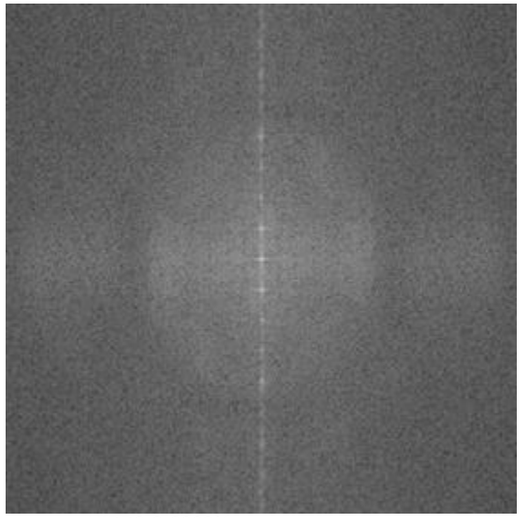
1) Apply DFT

2) Use thresholding to recover principal direction of intensity change

3) Fit a straight line

4) Rotate spatial image





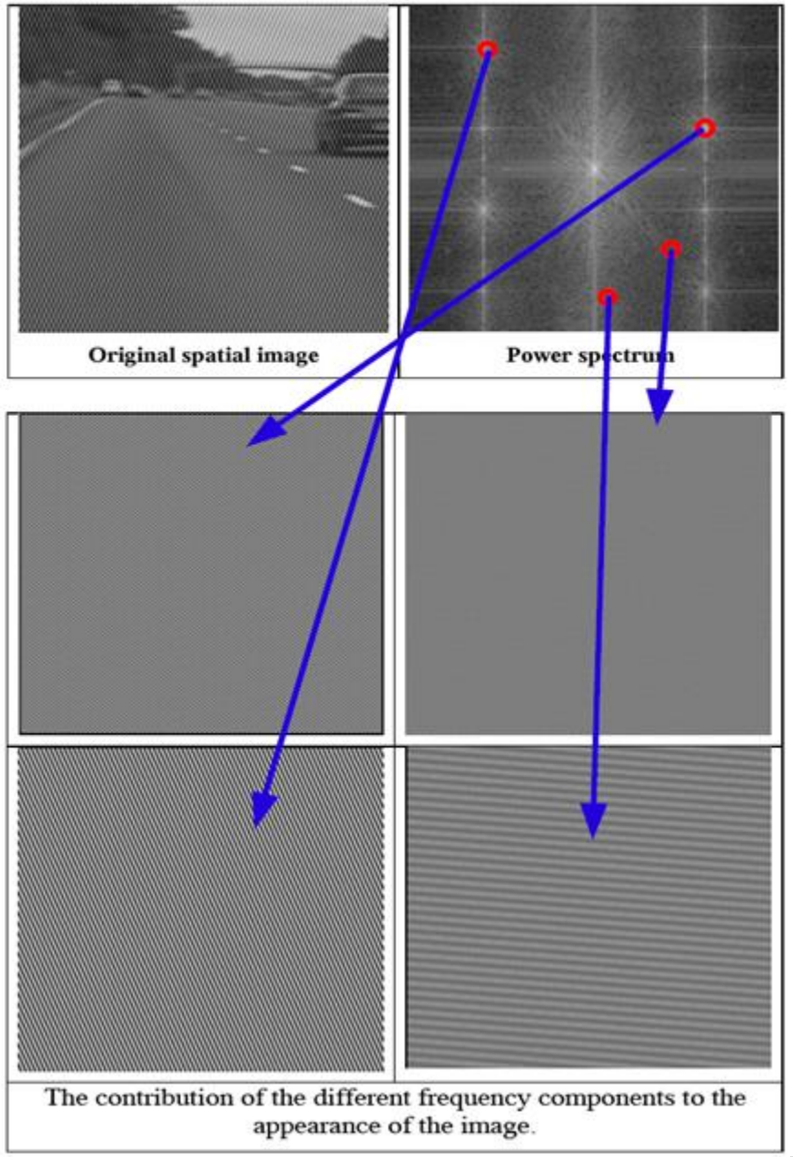
source: HIPR2 © 2003/4

DFT Application: Fourier Space Filtering

- Suppressing certain frequencies whilst leaving others unchanged.
  (will be covered later)

Example:

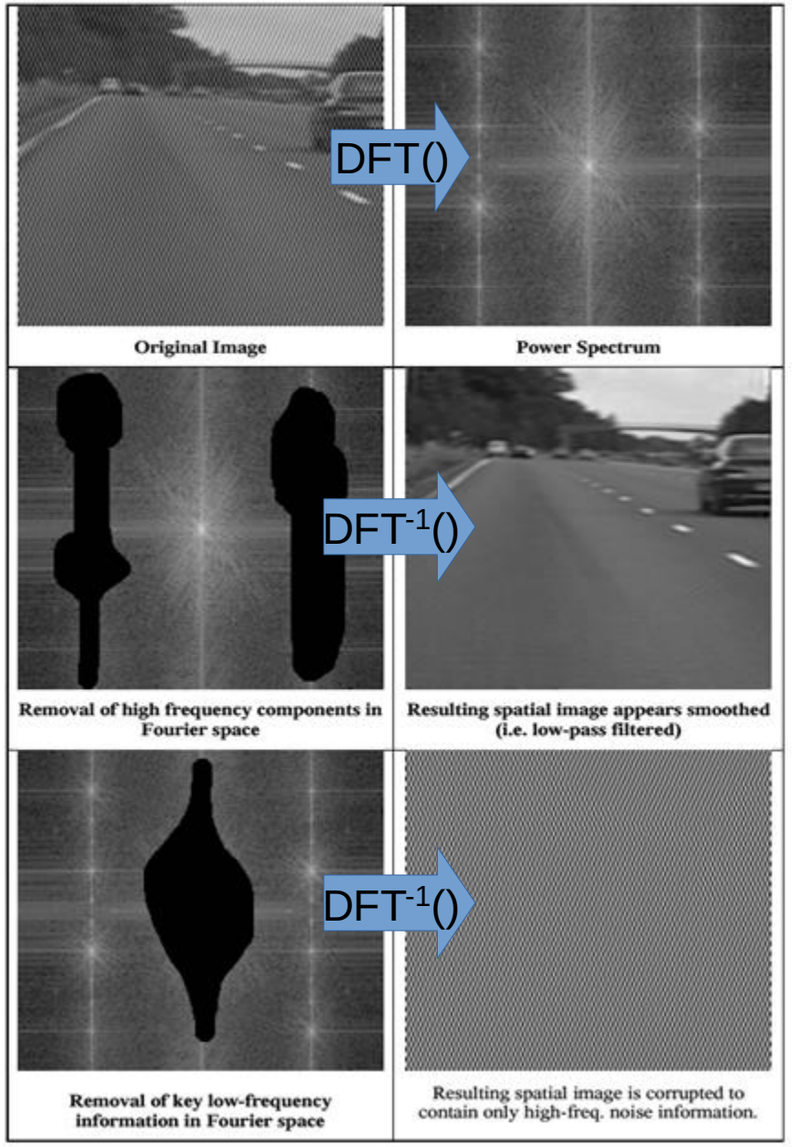
- Filtering using the Amplitudes.



DFT Application: Fourier Space Filtering

Example:

- First compute and inspect the amplitude spectrum of the image
- Then hand-craft a binary mask *P*
  - 1 to keep a frequency
  - 0 to remove that frequency
- component wise multiplication of amplitude spectrum and mask *P*
- inverse Fourier transform.



DFT Application: Fourier Space Filtering

Removing frequencies in the Fourier domain by setting their magnitude to zero results in frequency filtering in the spatial domain.

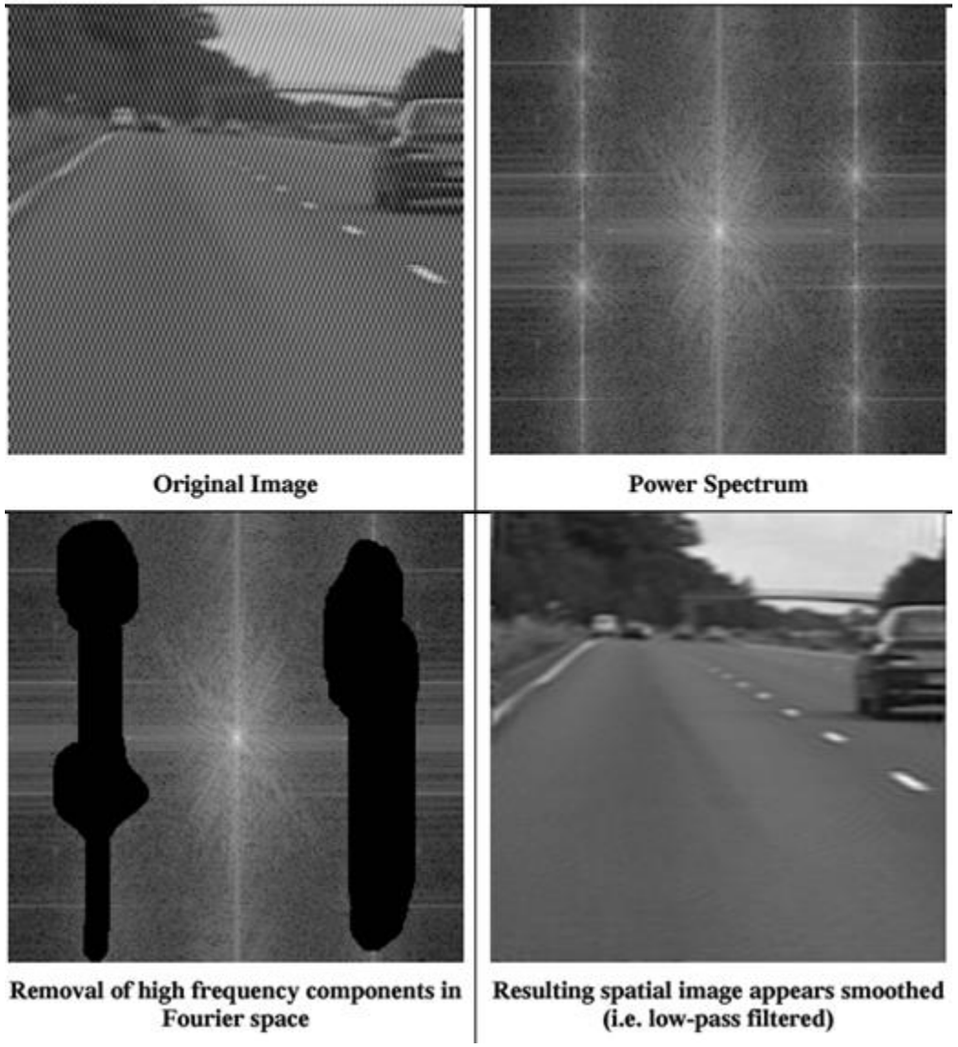


Image Frequencies - Example

More frequencies are iteratively added (from the lowest to the highest).

(lowest 5, 10, 20, 40, 80, and 160 spatial frequency elements of the original)

Image Frequencies - Example

More frequencies are iteratively added (from the highest to the lowest).

(highest 5, 10, 20, 40, 80, and 160 spatial frequency elements of the original)

Reminder: Discrete Convolution Theorem

Convolution in spatial domain reduces to multiplication in frequency domain!

$$I_A * I_B = \mathrm{DFT}^{-1}(\mathrm{DFT}(I_A) \cdot \mathrm{DFT}(I_B))$$

where $*$ denotes convolution and $\cdot$ element-wise multiplication of two matrices.

…also convolution in the frequency domain (Fourier space) reduces to multiplication in the spatial domain (Real space)

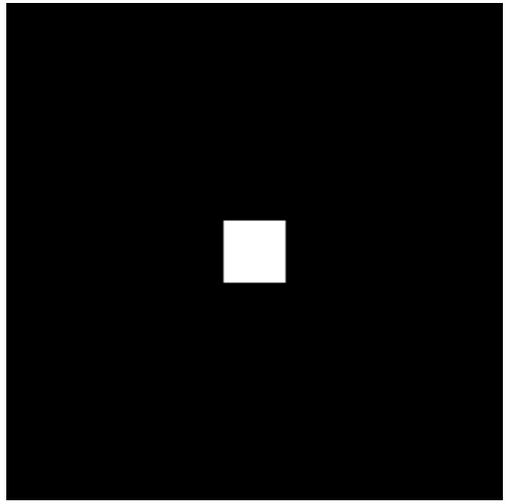
DFT Application: Fast Filtering

- Efficient implementation of spatial filtering:
  - Make image and mask the same dimension by zero padding.
  - Apply FFT to the image and to the mask.
  - Perform element-wise multiplication of the corresponding Fourier spectra.
  - Return to the spatial domain by applying inverse FFT.
- If the mask is large, the algorithm is more efficient than doing the convolution directly in the spatial domain.
- Some image processing / computer vision libraries switch automatically to this algorithm when the mask is large.

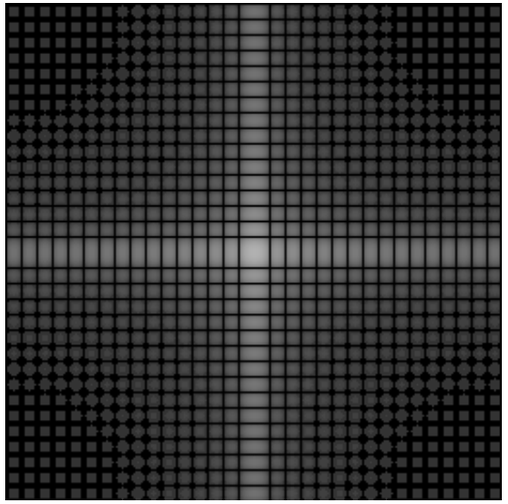
DFT Application: Filter Analysis

Using the convolution theorem, predict the behaviour of a filter against variations in the frequency of the noise.

The behaviour of the mean filter against variations in noise frequency is erratic.



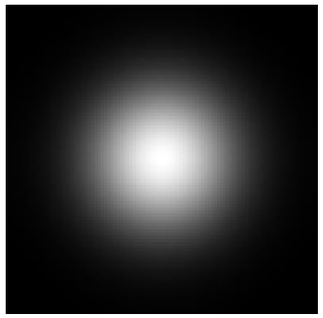
Mean filter
(zero padded)



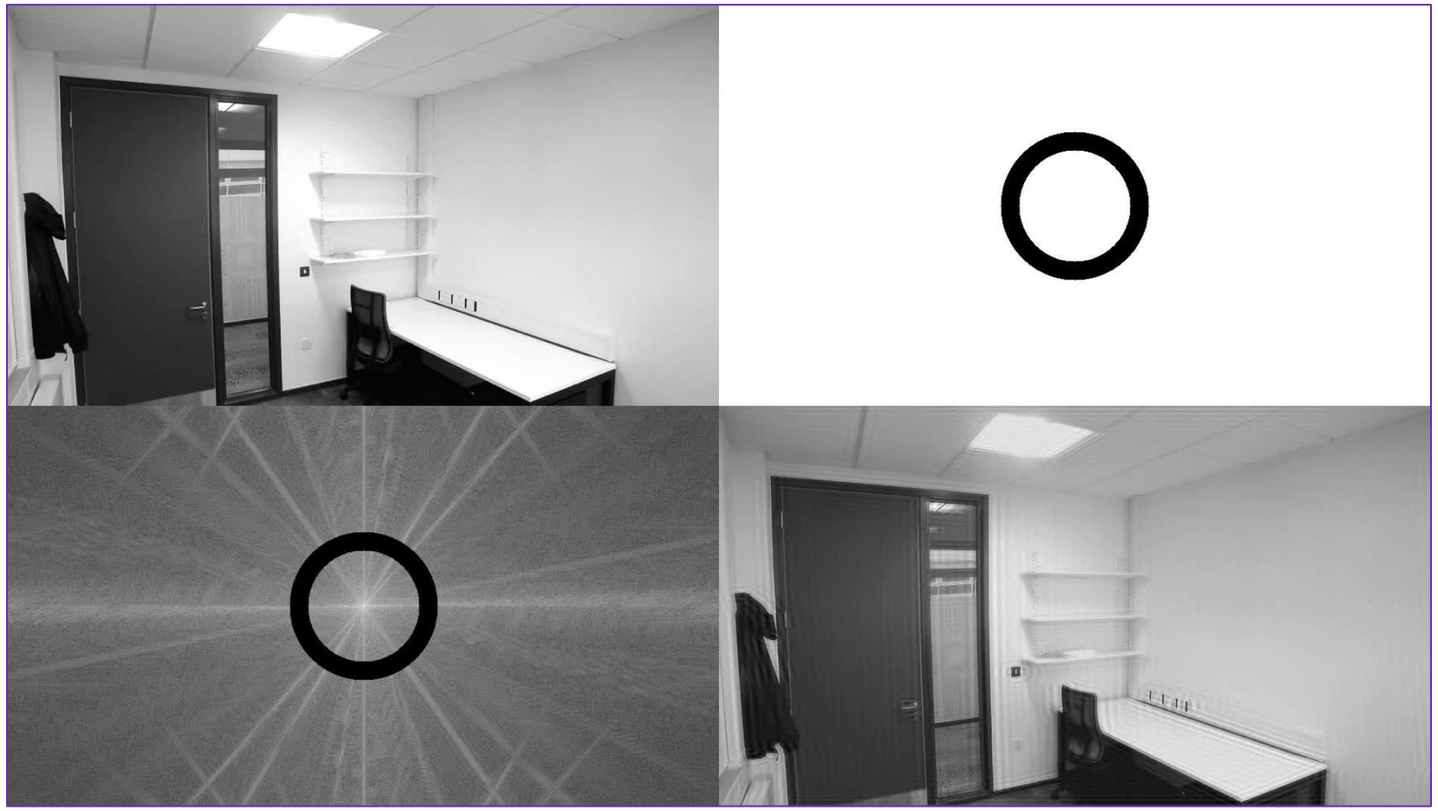
Magnitude of the Fourier spectrum
(Mean Filter)

The Fourier transform of a Gaussian is again a Gaussian – smooth and monotonic.

The Gaussian filter is robust against noise frequency variations.



Application: Filtering - Next Lecture Teaser



https://youtu.be/iPnSDdnlC8k

Band pass filtering, we maintain a zone of frequencies (more in Lecture 8!)

Quick Demonstration

- Fourier Transform:

https://github.com/atapour/ip-python-opencv/blob/main/fourier.py

- Band-Pass Filtering:

https://github.com/atapour/ip-python-opencv/blob/main/bandpass-filter-fourier.py

Summary

Fourier transform background.

DFT on images:

- visualising DFT
- understanding DFT
- the convolution theorem

Applications

- noise filtering
- fast convolution
- filter analysis